

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Operating Systems

COURSE CODE: CSA-0404

Name : BP VENKATA UDAY KUMAR

Reg no: 192411203

COURSE OUTCOME COVERED

CO3: Evaluate memory management mechanisms such as linking, paging, and segmentation to determine their effectiveness for different system requirements

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks: 100

CASE STUDY:

1. Describe the complete address translation mechanism using base and limit registers in the given system. Explain how this method ensures memory protection and prevents processes from accessing unauthorized memory locations.

2. Explain the concept of paging as implemented in the system. Describe how logical memory is divided into pages and mapped to physical frames. Discuss how paging helps in avoiding external fragmentation and improving memory utilization.

3. Describe the working of the Translation Lookaside Buffer (TLB) in the given system. Explain how TLB reduces memory access time and improves overall system performance.

4. Differentiate between TLB hit and TLB miss in detail. Explain how each case affects memory access time and system efficiency, and illustrate with an example.

5. Describe the concept of segmentation in detail. Explain how programs are divided into logical segments such as code, data, and stack, and discuss the benefits of segmentation over other memory management techniques.

6. Explain the structure and functioning of a segment table. Describe how logical addresses are translated into physical addresses using segment number and offset.

7.
Compare and contrast paging and segmentation in detail. Discuss their differences in terms of memory allocation, fragmentation, flexibility, and performance.

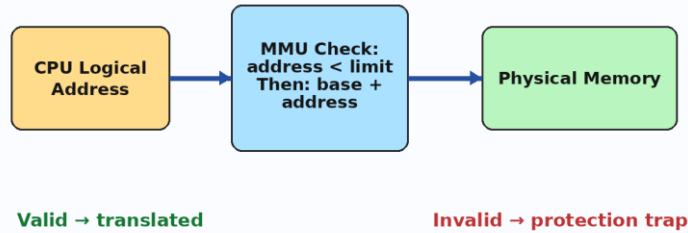
8. Explain the concept of virtual memory and its importance in modern operating systems. Describe how it allows execution of programs larger than the available physical memory.

9. Describe the page fault handling process in detail. Explain each step involved when a page is not found in memory and how the operating system resolves the fault.

10. Explain the need for page replacement algorithms in virtual memory systems. Describe the working of the FIFO page replacement algorithm with a suitable example.

1. Complete Address Translation Using Base and Limit Registers

1. Base + Limit Address Translation



Base and limit registers provide a simple method of address translation, relocation, and memory protection. The base register stores the starting physical address assigned to the currently executing process. The limit register stores the size of the process's legal logical address space. Every address generated by a user program is treated as a logical address or offset.

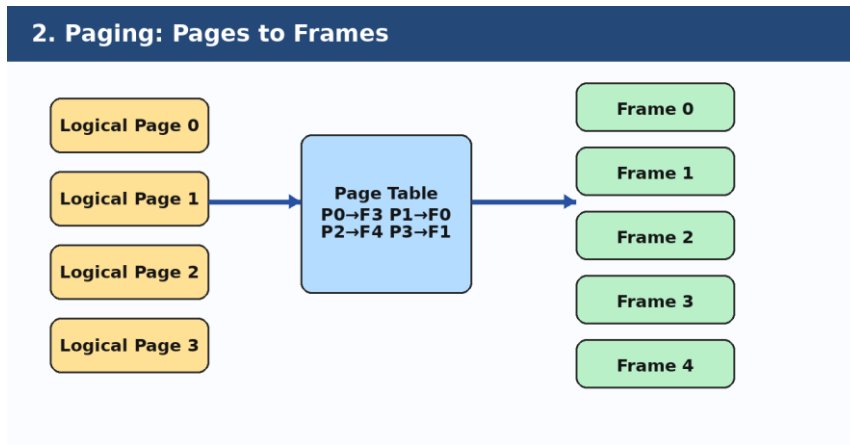
When the CPU generates a logical address, the Memory Management Unit (MMU) first compares it with the limit register. If $0 \leq \text{logical address} < \text{limit}$, the reference is legal. The MMU then calculates the physical address using: $\text{Physical Address} = \text{Base Register} + \text{Logical Address}$. For example, if base = 14,000, limit = 4,000, and the CPU generates logical address 850, the address is valid and the physical address is 14,850.

If the process generates logical address 4,250, it exceeds the limit. The MMU blocks the access and generates a protection fault or trap to the operating system. Thus, an invalid address never reaches physical memory.

This mechanism ensures memory protection because each process can access only the memory range allocated to it. Ordinary user programs cannot modify base and limit registers; only the operating system can load them using privileged instructions. During a context switch, the OS loads the correct values for the next process. Therefore, one process cannot access another process's memory or protected OS memory.

The method is fast because translation requires only a bounds comparison and an addition. It also supports relocation: a program can be moved to another physical location simply by changing the base register. Its main limitation is that a process normally needs one contiguous physical-memory region, which can cause external fragmentation.

2. Paging



Paging is a memory-management technique in which logical memory is divided into fixed-size blocks called pages and physical memory is divided into equal-size blocks called frames. Because pages and frames have the same size, any page of a process can be placed in any available physical frame.

A logical address is divided into a page number and page offset. The page number identifies the required logical page, while the offset specifies the exact position within that page. Each process has a page table maintained by the operating system. The page table maps every logical page number to a physical frame number.

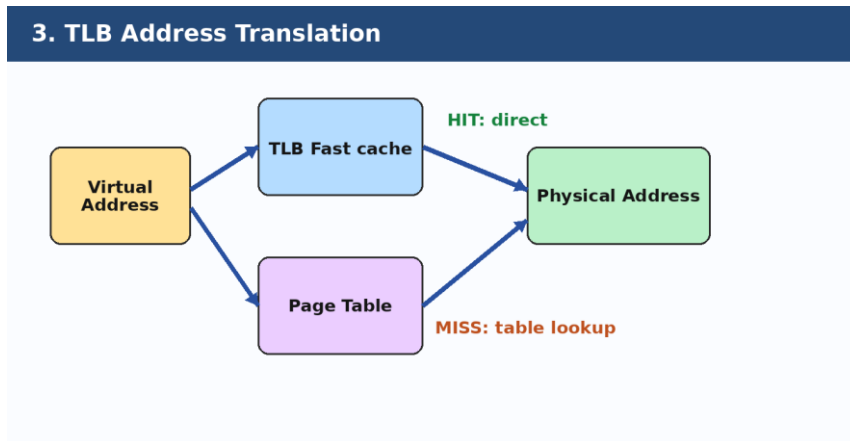
For example, assume the page size is 1 KB and logical address is 2,500. Page number = $2500 // 1024 = 2$ and offset = 452. If page 2 is stored in frame 7, the physical address is $(7 \times 1024) + 452 = 7,620$. The MMU performs this translation automatically.

The major advantage of paging is that a process does not need contiguous physical memory. Page 0 may be in frame 5, page 1 in frame 12, and page 2 in frame 7. This eliminates external fragmentation because any free frame can be used for any page. It also makes allocation and deallocation easier and improves utilization of scattered free memory.

Paging may still cause internal fragmentation, especially when the final page is not completely filled. It also requires page tables, which consume memory and add translation overhead. Modern systems reduce this overhead using a TLB.

Paging is also the foundation of virtual memory. Only the pages currently required need to remain in RAM; other pages can be stored on secondary storage and loaded when needed. Page-level protection bits can control reading, writing, execution, and sharing.

3. Translation Lookaside Buffer (TLB)



The Translation Lookaside Buffer (TLB) is a small, high-speed associative cache used by the Memory Management Unit to store recently used virtual-page-to-physical-frame translations. Its main purpose is to reduce the time required for address translation in a paged memory system.

Normally, when the CPU generates a virtual address, the system must consult the page table to determine the corresponding physical frame. If the page table is stored in main memory, this can require an additional memory access before the actual data is accessed. The TLB avoids many of these page-table accesses.

The CPU divides the virtual address into a page number and offset. The MMU searches the TLB for the page number. If the mapping is present, a TLB hit occurs. The corresponding frame number is obtained immediately and combined with the offset to form the physical address. The page table in RAM is not consulted.

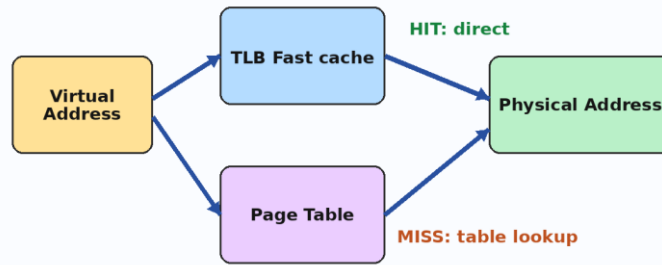
If the mapping is absent, a TLB miss occurs. The system reads the page-table entry. If the page is resident in RAM, the translation is loaded into the TLB and the memory access continues. A TLB miss is not necessarily a page fault; a page fault occurs only when the required page itself is absent from physical memory.

For example, if page 6 maps to frame 21 and this entry is in the TLB, a reference to page 6 can be translated immediately. With a 4 KB page and offset 300, the physical address is $(21 \times 4096) + 300 = 86,316$.

The TLB works well because programs exhibit locality of reference. Recently used pages are likely to be used again. A high TLB hit ratio reduces effective memory-access time, decreases CPU stalls, reduces page-table traffic, and improves overall system performance.

4. TLB Hit and TLB Miss

4. TLB Hit vs TLB Miss



A TLB hit occurs when the required virtual-page-to-frame mapping is already present in the Translation Lookaside Buffer. The CPU generates a virtual address, the MMU searches the TLB, finds a matching entry, obtains the frame number, combines it with the offset, and accesses memory. Because the page table does not need to be accessed, this is the faster case.

Example: page size = 4 KB, virtual page = 3, offset = 100, and the TLB contains page 3 → frame 9. The physical address is $(9 \times 4096) + 100 = 36,964$. If TLB lookup takes 10 ns and memory access takes 100 ns, an illustrative hit time is approximately 110 ns.

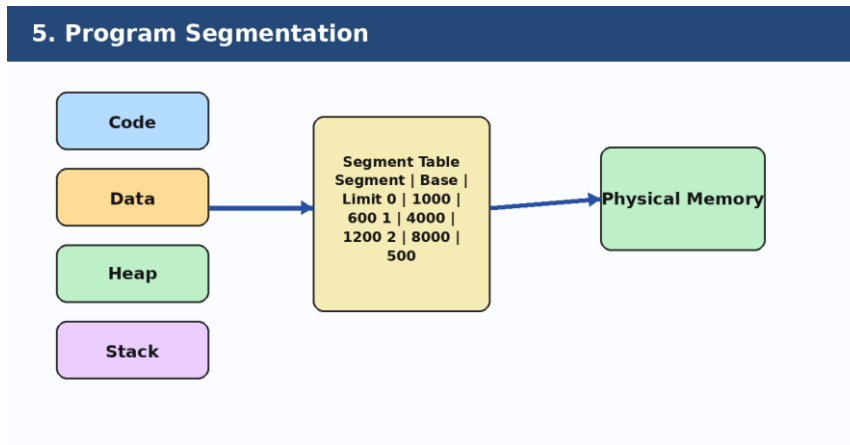
A TLB miss occurs when the required mapping is not available in the TLB. The MMU must consult the page table to obtain the frame number. If the page-table entry is valid and the page is present in RAM, the mapping is added to the TLB and the original access continues. In a simple system, this may require approximately 10 ns for TLB lookup + 100 ns for page-table access + 100 ns for data access = 210 ns.

A TLB miss is different from a page fault. During a TLB miss, the required page may already be in physical memory. A page fault means the page is not resident and the OS must bring it into RAM.

The hit ratio strongly affects performance. With a 98% hit ratio, most translations follow the fast path. With a low hit ratio, the CPU performs more expensive page-table walks and waits longer for memory.

Therefore, TLB hits improve system efficiency, while TLB misses increase address-translation overhead. Modern processors use larger or multi-level TLBs, address-space identifiers, huge pages, and page-walk caches to maintain high translation performance.

5. Segmentation



Segmentation is a memory-management technique that divides a program according to its logical structure. Instead of fixed-size pages, a process is divided into variable-sized segments such as code, data, heap, stack, procedures, libraries, and tables. Each segment represents a meaningful part of the program.

A logical address in segmentation consists of two components: segment number and offset. The segment number identifies the required logical segment and the offset identifies a location within it. For example, segment 0 may contain program code, segment 1 data, segment 2 heap, and segment 3 stack.

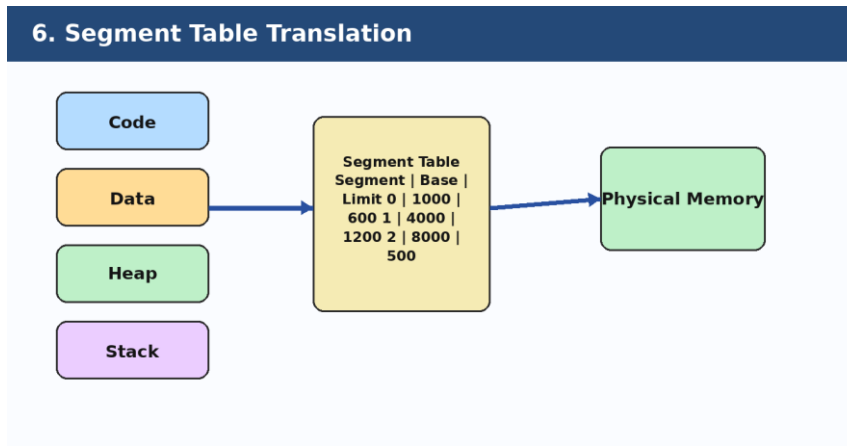
Each process has a segment table. Every segment-table entry stores the segment's base physical address and limit or length. When the CPU generates <segment number, offset>, the MMU selects the segment-table entry and verifies that the offset is smaller than the segment limit. If valid, $\text{Physical Address} = \text{Segment Base} + \text{Offset}$. If invalid, a protection exception occurs.

Segmentation offers strong logical protection. Different permissions can be assigned to different segments. Code may be read and executed but not modified, while data and stack may be readable and writable. Shared libraries or common code can also be mapped into multiple processes while private data remains separate.

Another benefit is flexibility. The stack and heap can grow independently, and program modules can be managed separately. Segmentation therefore matches the programmer's logical view better than fixed-size paging.

Its major disadvantage is external fragmentation. Because segments have variable sizes and each segment generally requires contiguous physical space, free memory can become divided into small holes. Compaction may be needed. Despite this limitation, segmentation is important for understanding logical memory organization, sharing, relocation, and protection.

6. Segment Table



A segment table is the main data structure used for address translation in a segmented memory-management system. Each process has a number of logical segments, and the segment table contains an entry describing each segment.

A segment-table entry normally contains a base address and limit. The base indicates where the segment begins in physical memory. The limit specifies the size of the segment or maximum valid offset. An entry may also contain valid, read, write, execute, sharing, and privilege information.

A logical address is represented as $\langle \text{segment number}, \text{offset} \rangle$. First, the segment number is used as an index into the segment table. The system verifies that the segment number itself is valid. The MMU then obtains the base and limit from the selected entry.

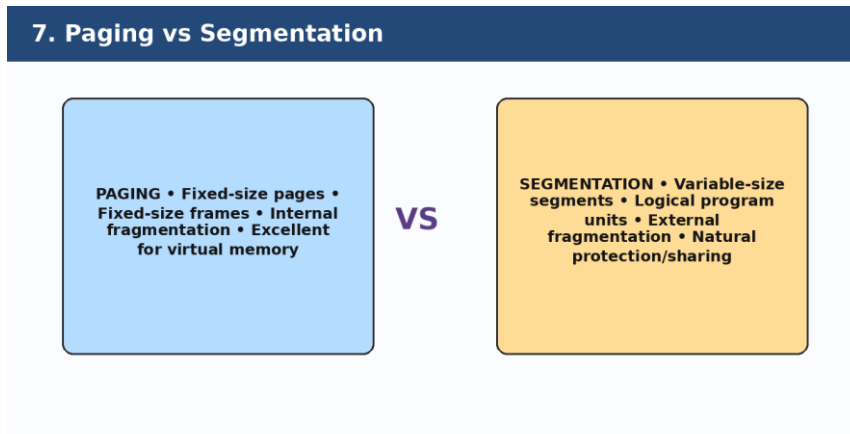
Next, the offset is compared with the limit. If the offset is outside the segment, the MMU generates a segmentation or protection fault. If it is valid, the physical address is calculated using: $\text{Physical Address} = \text{Segment Base} + \text{Offset}$.

For example, suppose segment 0 has base 1000 and limit 600, segment 1 has base 4000 and limit 1200, and segment 2 has base 8000 and limit 500. Logical address $\langle 1, 350 \rangle$ is valid because $350 < 1200$. Its physical address is $4000 + 350 = 4350$. However, $\langle 2, 700 \rangle$ is invalid because 700 exceeds segment 2's limit.

The operating system controls segment-table information. User processes cannot arbitrarily modify these mappings. During a context switch, the OS activates the table belonging to the scheduled process.

Thus, the segment table performs four important functions: relocation through the base value, bounds checking through the limit, protection through permission bits, and logical organization through separate entries. It enables code, data, stack, and other program regions to be managed and protected independently.

7. Paging and Segmentation – Comparison



Paging and segmentation are memory-management techniques that map logical addresses to physical memory, but they organize memory differently. Paging divides memory into fixed-size pages and frames, whereas segmentation divides a program into variable-size logical units such as code, data, heap, and stack.

In paging, the logical address contains a page number and offset. The page number indexes a page table to obtain a physical frame number. Pages can be stored in any available frames, so the process does not need contiguous physical memory. Paging is normally invisible to the programmer.

In segmentation, the logical address contains a segment number and offset. The segment number selects an entry containing a base and limit. If the offset is valid, the physical address is $\text{base} + \text{offset}$. Segments correspond directly to logical program components and may have different sizes.

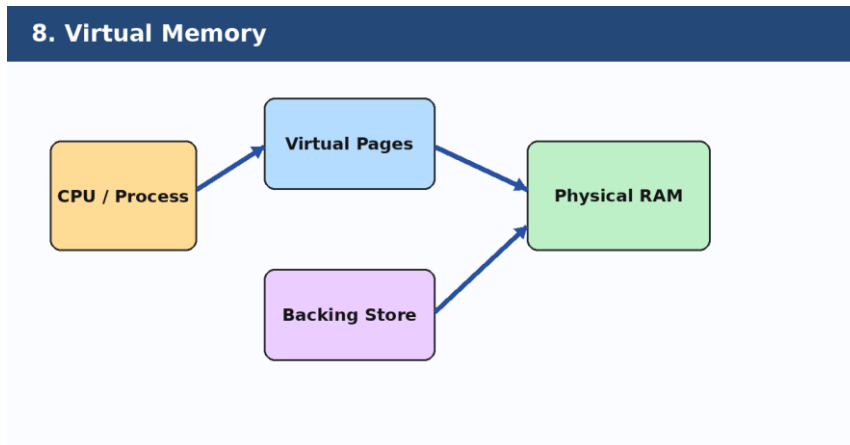
Fragmentation differs significantly. Paging eliminates external fragmentation because every frame has the same size, but it can cause internal fragmentation when a page is not fully used. Segmentation has little fixed-block internal waste but can cause external fragmentation because variable-size segments leave holes in physical memory.

Paging is particularly effective for virtual memory because individual fixed-size pages can be loaded, replaced, and stored easily. Segmentation is particularly useful for logical protection and sharing because permissions can naturally be assigned to complete program units.

Performance also differs. Paging uses page-table lookups and commonly a TLB. Segmentation requires segment-table lookup, limit checking, and base addition. Hardware support makes both techniques efficient.

In summary: paging uses fixed sizes, supports non-contiguous allocation, causes mainly internal fragmentation, and is ideal for demand-paged virtual memory. Segmentation uses variable logical units, offers natural modular protection and sharing, but can cause external fragmentation. Some systems combine the two ideas to gain advantages from both.

8. Virtual Memory



Virtual memory is a memory-management technique that gives each process a large, private, continuous address space even though only part of it may be present in physical RAM at a given time. The remaining pages can be stored on secondary storage and loaded when required.

The CPU generates virtual addresses rather than direct physical addresses. The MMU translates these addresses through page tables, usually accelerated by a TLB. Each page-table entry indicates whether the page is currently resident in a physical frame. If present, the access proceeds normally. If absent but valid, a page fault occurs.

Demand paging is the common virtual-memory approach. A page is loaded into RAM only when the process actually references it. Programs exhibit locality, so only a relatively small working set of instructions and data may need to be resident at one time.

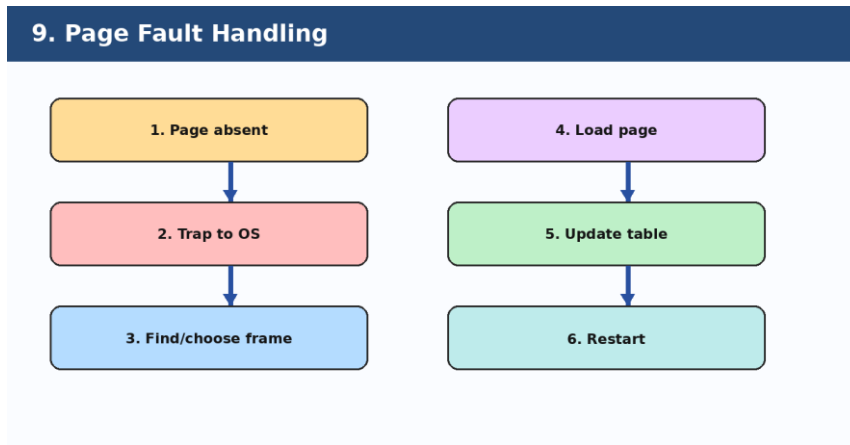
This allows programs larger than available physical memory to execute. For example, a computer with 8 GB of RAM can provide a process with a much larger virtual address space because inactive portions do not need to occupy RAM simultaneously. It also allows many processes to execute concurrently by sharing physical frames efficiently.

Virtual memory provides isolation because each process has its own address mappings. It also supports protection, shared libraries, memory-mapped files, and copy-on-write. The same physical page can be intentionally mapped into multiple processes when controlled sharing is required.

The main disadvantage is page-fault cost. Accessing secondary storage is much slower than RAM. If too many actively needed pages are absent, the system may spend excessive time swapping pages, a condition called thrashing.

Virtual memory is therefore essential in modern operating systems. It provides large address spaces, efficient physical-memory utilization, process isolation, protection, controlled sharing, and the ability to run applications whose total memory requirements exceed installed RAM.

9. Page Fault Handling Process



A page fault occurs when a process references a valid virtual page that is not currently present in physical memory. The MMU detects the not-present page-table entry and raises a page-fault exception, transferring control from the process to the operating system.

Step 1 – Detect the fault: The MMU finds that the required page is absent. The CPU saves the necessary process state and enters kernel mode.

Step 2 – Validate the address: The OS checks whether the virtual address belongs to a valid process region and whether the requested read, write, or execute operation is permitted. An illegal reference may terminate the process.

Step 3 – Locate the page: If the reference is valid, the OS determines where the page contents are stored, such as swap space, an executable file, or a memory-mapped file.

Step 4 – Obtain a frame: The OS searches for a free physical frame. If none is available, it selects a victim page using a page-replacement algorithm such as FIFO, Clock, or LRU approximation.

Step 5 – Handle the victim: If the victim page has been modified, it may need to be written to secondary storage before its frame is reused.

Step 6 – Load the required page: The OS reads the requested page into the selected frame. The faulting process normally waits while this I/O operation occurs.

Step 7 – Update tables: The page-table entry is updated with the new frame number and marked present. Protection and status information is set, and any stale TLB entry is corrected.

Step 8 – Restart execution: The OS restores the process state and restarts the instruction that caused the fault. The memory reference can now complete normally.

This mechanism makes demand paging transparent to applications, but page faults are expensive, so operating systems attempt to keep frequently used pages in RAM.

10. Need for Page Replacement and FIFO Algorithm

10. FIFO Page Replacement

Reference string: 2 3 4 1 2 5 1 2 3 4 5



FIFO removes the page that entered memory first.

Page replacement is required in virtual-memory systems when a page fault occurs and no free physical frame is available. The operating system must remove one resident page to create space for the required page. A good replacement algorithm attempts to reduce future page faults while keeping management overhead low.

FIFO (First-In First-Out) is one of the simplest page-replacement algorithms. It treats pages in memory like a queue. The page that entered memory first is considered the oldest and is selected for replacement when a new page must be loaded. New pages are added to the rear of the queue.

Example: consider three frames and the reference string:
1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5.

Initially, references 1, 2, and 3 cause faults and fill the frames: [1,2,3]. Reference 4 faults and replaces the oldest page 1, producing [4,2,3]. Reference 1 faults and replaces 2: [4,1,3]. Reference 2 faults and replaces 3: [4,1,2]. Reference 5 faults and replaces 4: [5,1,2]. The next references 1 and 2 are hits. Reference 3 faults and replaces 1: [5,3,2]. Reference 4 faults and replaces 2: [5,3,4]. The final reference 5 is a hit.

For this example, FIFO produces 9 page faults. FIFO is easy to understand and implement because only page arrival order must be maintained.

Its disadvantage is that it ignores how frequently or recently a page is used. An important page may be removed simply because it is old. FIFO can also exhibit Belady's anomaly, where adding more frames unexpectedly increases page faults for some reference strings.

Thus, page replacement allows virtual memory to continue operating when RAM is full, while FIFO provides a simple oldest-page-first replacement policy.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning

Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand